

- 1) Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

```
mysql> INSERT INTO Customer VALUES
  -> (101, 'Arun'),
  -> (102, 'Kumar');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
  -> (201, 'Laptop', 55000.00),
  -> (202, 'Mouse', 800.00);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Orders VALUES
  -> (1, 101),
  -> (2, 102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO OrderItem VALUES
  -> (1, 201, 1),
  -> (1, 202, 2),
  -> (2, 201, 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display tables
mysql> SELECT * FROM Customer;
+-----+-----+
| CustID | CustName |
+-----+-----+
|      101 | Arun      |
|      102 | Kumar     |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
|      201 | Laptop   | 55000.00 |
|      202 | Mouse    | 800.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Orders;
+-----+-----+
| OrderID | CustID |
+-----+-----+
|        1 |    101 |
|        2 |    102 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM OrderItem;
+-----+-----+-----+
| OrderID | ProdID | Qty |
+-----+-----+-----+
|        1 |    201 | 1 |
|        1 |    202 | 2 |
|        2 |    201 | 1 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql>
mysql> -- Display complete order details
mysql> SELECT o.OrderID, c.CustID, c.CustName,
->      p.ProdID, p.ProdName, oi.Qty, p.Price
-> FROM Orders o
-> JOIN Customer c ON o.CustID = c.CustID
-> JOIN OrderItem oi ON o.OrderID = oi.OrderID
-> JOIN Product p ON oi.ProdID = p.ProdID;
+-----+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+-----+-----+
|        2 |    102 | Kumar    |    201 | Laptop   | 1 | 55000.00 |
|        1 |    101 | Arun     |    201 | Laptop   | 1 | 55000.00 |
|        1 |    101 | Arun     |    202 | Mouse    | 2 | 800.00 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

2) A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

```
mysql>
mysql> -- Add foreign keys
mysql> ALTER TABLE Patient
    -> ADD CONSTRAINT fk_patient_doctor
    -> FOREIGN KEY (DocID) REFERENCES Doctor(DocID);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> ALTER TABLE Patient
    -> ADD CONSTRAINT fk_patient_ward
    -> FOREIGN KEY (WardNo) REFERENCES Ward(WardNo);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Sample data
mysql> INSERT INTO Doctor VALUES
    -> (501, 'Dr. Ravi', 'Cardiology'),
    -> (502, 'Dr. Priya', 'Neurology');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Ward VALUES
    -> (1, 'General Ward'),
    -> (2, 'ICU');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Patient VALUES
    -> (1001, 'Arun', 501, 1),
    -> (1002, 'Kumar', 502, 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> -- Display
mysql> SELECT * FROM Doctor;
```

DocID	DocName	Specialization
501	Dr. Ravi	Cardiology
502	Dr. Priya	Neurology

```
2 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM Ward;
```

WardNo	WardName
1	General Ward
2	ICU

```
2 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM Patient;
```

PID	PName	DocID	WardNo
1001	Arun	501	1
1002	Kumar	502	2

```
2 rows in set (0.00 sec)
```

```
mysql>
mysql> -- Complete patient details
mysql> SELECT p.PID, p.PName,
->      d.DocID, d.DocName, d.Specialization,
->      w.WardNo, w.WardName
-> FROM Patient p
-> JOIN Doctor d ON p.DocID = d.DocID
-> JOIN Ward w ON p.WardNo = w.WardNo;
```

PID	PName	DocID	DocName	Specialization	WardNo	WardName
1001	Arun	501	Dr. Ravi	Cardiology	1	General Ward
1002	Kumar	502	Dr. Priya	Neurology	2	ICU

```
2 rows in set (0.01 sec)
```

- 3) A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

```
mysql>
mysql> -- Sample data
mysql> INSERT INTO Student VALUES
    -> (101, 'Arun'),
    -> (102, 'Kumar');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Course VALUES
    -> (501, 'DBMS', 'Dr. Ravi'),
    -> (502, 'Java', 'Dr. Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Enrollment VALUES
    -> (101, 501, 'A'),
    -> (101, 502, 'B+'),
    -> (102, 501, 'A+');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display
mysql> SELECT * FROM Student;
+-----+-----+
| SID | SName |
+-----+-----+
| 101 | Arun  |
| 102 | Kumar |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Course;
```

CourseID	CourseName	Faculty
501	DBMS	Dr. Ravi
502	Java	Dr. Priya

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Enrollment;
```

SID	CourseID	Grade
101	501	A
101	502	B+
102	501	A+

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Complete enrollment details
```

```
mysql> SELECT s.SID, s.SName,  
->      c.CourseID, c.CourseName, c.Faculty,  
->      e.Grade  
-> FROM Enrollment e  
-> JOIN Student s ON e.SID = s.SID  
-> JOIN Course c ON e.CourseID = c.CourseID;
```

SID	SName	CourseID	CourseName	Faculty	Grade
102	Kumar	501	DBMS	Dr. Ravi	A+
101	Arun	501	DBMS	Dr. Ravi	A
101	Arun	502	Java	Dr. Priya	B+

```
3 rows in set (0.00 sec)
```

- 4) Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

```
mysql>
mysql> -- Sample data
mysql> INSERT INTO Flight VALUES
    -> ('AI101', 'Chennai', 'Delhi'),
    -> ('AI102', 'Mumbai', 'Bangalore');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Passenger VALUES
    -> (1001, 'Arun'),
    -> (1002, 'Kumar');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Booking VALUES
    -> ('AI101', 1001, '2026-08-13', '12A'),
    -> ('AI101', 1002, '2026-08-13', '12B'),
    -> ('AI102', 1001, '2026-08-14', '10C');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display
mysql> SELECT * FROM Flight;
+-----+-----+-----+
| FlightNo | DeptCity | ArrCity |
+-----+-----+-----+
| AI101    | Chennai  | Delhi   |
| AI102    | Mumbai   | Bangalore |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Passenger;
```

PassID	PassName
1001	Arun
1002	Kumar

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Booking;
```

FlightNo	PassID	BookingDate	Seat
AI101	1001	2026-08-13	12A
AI101	1002	2026-08-13	12B
AI102	1001	2026-08-14	10C

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Complete booking details
```

```
mysql> SELECT b.FlightNo,  
->      p.PassID,  
->      p.PassName,  
->      b.Seat,  
->      f.DeptCity,  
->      f.ArrCity,  
->      b.BookingDate  
-> FROM Booking b  
-> JOIN Passenger p ON b.PassID = p.PassID  
-> JOIN Flight f ON b.FlightNo = f.FlightNo;
```

FlightNo	PassID	PassName	Seat	DeptCity	ArrCity	BookingDate
AI101	1002	Kumar	12B	Chennai	Delhi	2026-08-13
AI101	1001	Arun	12A	Chennai	Delhi	2026-08-13
AI102	1001	Arun	10C	Mumbai	Bangalore	2026-08-14

```
3 rows in set (0.00 sec)
```


- 5) Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

```
mysql>
mysql> -- Sample data
mysql> INSERT INTO Department VALUES
    -> (10, 'IT'),
    -> (20, 'HR');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Manager VALUES
    -> (501, 'Ravi'),
    -> (502, 'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Job VALUES
    -> (101, 'Software Engineer', 60000),
    -> (102, 'HR Executive', 50000);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Employee VALUES
    -> (1001, 'Arun', 10, 501, 101),
    -> (1002, 'Kumar', 20, 502, 102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
|      10 | IT       |
|      20 | HR       |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Manager;
```

ManagerID	ManagerName
501	Ravi
502	Priya

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Job;
```

JobID	JobTitle	Salary
101	Software Engineer	60000.00
102	HR Executive	50000.00

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Employee;
```

EmpID	EmpName	DeptID	ManagerID	JobID
1001	Arun	10	501	101
1002	Kumar	20	502	102

```
2 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Complete employee details
```

```
mysql> SELECT e.EmpID, e.EmpName,  
-> d.DeptName,  
-> m.ManagerName,  
-> j.JobTitle, j.Salary  
-> FROM Employee e  
-> JOIN Department d ON e.DeptID = d.DeptID  
-> JOIN Manager m ON e.ManagerID = m.ManagerID  
-> JOIN Job j ON e.JobID = j.JobID;
```

EmpID	EmpName	DeptName	ManagerName	JobTitle	Salary
1001	Arun	IT	Ravi	Software Engineer	60000.00
1002	Kumar	HR	Priya	HR Executive	50000.00

```
2 rows in set (0.00 sec)
```

6) Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

```
mysql>
mysql> -- Sample data
mysql> INSERT INTO Member VALUES
    -> (101, 'Arun'),
    -> (102, 'Kumar');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Book VALUES
    -> (201, 'Database Systems', 'Education'),
    -> (202, 'Operating Systems', 'Education');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Author VALUES
    -> (301, 'Ravi'),
    -> (302, 'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO BookAuthor VALUES
    -> (201, 301),
    -> (202, 302);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Borrowing VALUES
    -> (101, 201, '2026-08-01'),
    -> (101, 202, '2026-08-05');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> -- Display
mysql> SELECT * FROM Member;
+-----+-----+
| MemberID | MemberName |
+-----+-----+
|      101 | Arun       |
|      102 | Kumar      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Book;
+-----+-----+-----+
| BookID | Title                | Genre |
+-----+-----+-----+
|      201 | Database Systems    | Education |
|      202 | Operating Systems   | Education |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Author;
+-----+-----+
| AuthorID | AuthorName |
+-----+-----+
|      301 | Ravi       |
|      302 | Priya      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM BookAuthor;
+-----+-----+
| BookID | AuthorID |
+-----+-----+
|      201 |      301 |
|      202 |      302 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Borrowing;
+-----+-----+-----+
| MemberID | BookID | BorrowDate |
+-----+-----+-----+
|      101 |      201 | 2026-08-01 |
|      101 |      202 | 2026-08-05 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql>
mysql> -- Complete borrowing details
mysql> SELECT m.MemberID, m.MemberName,
->         b.BookID, b.Title, b.Genre,
->         br.BorrowDate
-> FROM Borrowing br
-> JOIN Member m ON br.MemberID = m.MemberID
-> JOIN Book b ON br.BookID = b.BookID;
```

MemberID	MemberName	BookID	Title	Genre	BorrowDate
101	Arun	201	Database Systems	Education	2026-08-01
101	Arun	202	Operating Systems	Education	2026-08-05

```
2 rows in set (0.03 sec)
```

7) Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

```

mysql>
mysql> -- Sample data
mysql> INSERT INTO Customer VALUES
    -> (101, 'Arun', '9876543210'),
    -> (102, 'Kumar', '9876543211');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Plan VALUES
    -> (1, 'Premium'),
    -> (2, 'Basic');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Billing VALUES
    -> (1001, 101, 1, '2026-08-01', 850.00),
    -> (1002, 102, 2, '2026-08-01', 450.00);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO CallDetails VALUES
    -> (5001, 1001, 120, 20.00),
    -> (5002, 1001, 300, 50.00),
    -> (5003, 1002, 180, 30.00);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display
mysql> SELECT * FROM Customer;
+-----+-----+-----+
| CustomerID | CustomerName | PhoneNo |
+-----+-----+-----+
|          101 | Arun          | 9876543210 |
|          102 | Kumar         | 9876543211 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

```
mysql> SELECT * FROM Plan;
```

PlanID	PlanName
1	Premium
2	Basic

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Billing;
```

BillID	CustomerID	PlanID	BillDate	TotalAmount
1001	101	1	2026-08-01	850.00
1002	102	2	2026-08-01	450.00

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM CallDetails;
```

CallID	BillID	CallDuration	CallCharge
5001	1001	120	20.00
5002	1001	300	50.00
5003	1002	180	30.00

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Billing report
```

```
mysql> SELECT b.BillID,  
->      c.CustomerID,  
->      c.CustomerName,  
->      c.PhoneNo,  
->      p.PlanName,  
->      b.BillDate,  
->      b.TotalAmount  
-> FROM Billing b  
-> JOIN Customer c ON b.CustomerID = c.CustomerID  
-> JOIN Plan p ON b.PlanID = p.PlanID;
```

BillID	CustomerID	CustomerName	PhoneNo	PlanName	BillDate	TotalAmount
1001	101	Arun	9876543210	Premium	2026-08-01	850.00
1002	102	Kumar	9876543211	Basic	2026-08-01	450.00

```
2 rows in set (0.00 sec)
```

- 8) A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

```
mysql> -- Sample data
mysql> INSERT INTO Supplier VALUES
    -> (1, 'ABC Suppliers'),
    -> (2, 'XYZ Suppliers');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Category VALUES
    -> (10, 'Electronics'),
    -> (20, 'Furniture');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Warehouse VALUES
    -> (100, 'Chennai Warehouse'),
    -> (101, 'Bangalore Warehouse');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
    -> (1001, 'Laptop', 1, 10, 55000, 100),
    -> (1002, 'Chair', 2, 20, 3500, 101);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display
mysql> SELECT * FROM Supplier;
+-----+-----+
| SupplierID | SupplierName |
+-----+-----+
|          1 | ABC Suppliers |
|          2 | XYZ Suppliers |
+-----+-----+
2 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM Category;
```

CategoryID	CategoryName
10	Electronics
20	Furniture

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Warehouse;
```

WarehouseID	WarehouseName
100	Chennai Warehouse
101	Bangalore Warehouse

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
```

PID	PName	SupplierID	CategoryID	Price	WarehouseID
1001	Laptop	1	10	55000.00	100
1002	Chair	2	20	3500.00	101

```
2 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Complete inventory
```

```
mysql> SELECT p.PID,  
->      p.PName,  
->      s.SupplierName,  
->      c.CategoryName,  
->      p.Price,  
->      w.WarehouseName  
-> FROM Product p  
-> JOIN Supplier s ON p.SupplierID = s.SupplierID  
-> JOIN Category c ON p.CategoryID = c.CategoryID  
-> JOIN Warehouse w ON p.WarehouseID = w.WarehouseID;
```

PID	PName	SupplierName	CategoryName	Price	WarehouseName
1001	Laptop	ABC Suppliers	Electronics	55000.00	Chennai Warehouse
1002	Chair	XYZ Suppliers	Furniture	3500.00	Bangalore Warehouse

```
2 rows in set (0.00 sec)
```

9) Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

```
mysql> -- Sample data
mysql> INSERT INTO Class VALUES
    -> (1, 'CSE-A'),
    -> (2, 'CSE-B');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Teacher VALUES
    -> (101, 'Dr. Ravi'),
    -> (102, 'Dr. Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Subject VALUES
    -> (201, 'DBMS', 101),
    -> (202, 'Operating Systems', 102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
    -> (1001, 'Arun', 1),
    -> (1002, 'Kumar', 2);
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Marks VALUES
    -> (1001, 201, 85),
    -> (1001, 202, 90),
    -> (1002, 201, 78);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> -- Display
mysql> SELECT * FROM Student;
+-----+-----+-----+
| SID | SName | ClassID |
+-----+-----+-----+
| 1001 | Arun  | 1       |
| 1002 | Kumar | 2       |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Class;
+-----+-----+
| ClassID | ClassName |
+-----+-----+
| 1       | CSE-A     |
| 2       | CSE-B     |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Teacher;
+-----+-----+
| TeacherID | TeacherName |
+-----+-----+
| 101       | Dr. Ravi    |
| 102       | Dr. Priya   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Subject;
+-----+-----+-----+
| SubjectID | SubjectName | TeacherID |
+-----+-----+-----+
| 201       | DBMS        | 101       |
| 202       | Operating Systems | 102       |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Marks;
+-----+-----+-----+
| SID | SubjectID | Marks |
+-----+-----+-----+
| 1001 | 201       | 85    |
| 1001 | 202       | 90    |
| 1002 | 201       | 78    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Complete student report
mysql> SELECT s.SID,
-> s.SName,
-> c.ClassName,
-> sub.SubjectName,
-> t.TeacherName,
-> m.Marks
-> FROM Marks m
-> JOIN Student s ON m.SID = s.SID
-> JOIN Class c ON s.ClassID = c.ClassID
-> JOIN Subject sub ON m.SubjectID = sub.SubjectID
-> JOIN Teacher t ON sub.TeacherID = t.TeacherID;
+-----+-----+-----+-----+-----+-----+
| SID | SName | ClassName | SubjectName | TeacherName | Marks |
+-----+-----+-----+-----+-----+-----+
| 1001 | Arun  | CSE-A     | DBMS        | Dr. Ravi    | 85    |
| 1001 | Arun  | CSE-A     | Operating Systems | Dr. Priya   | 90    |
| 1002 | Kumar | CSE-B     | DBMS        | Dr. Ravi    | 78    |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

- 10) Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

```
mysql> -- Sample data
mysql> INSERT INTO Movie VALUES
  -> (1, 'The Database Story'),
  -> (2, 'Digital World');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Actor VALUES
  -> (101, 'Arun Kumar'),
  -> (102, 'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Director VALUES
  -> (201, 'Ravi'),
  -> (202, 'Kumar');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Genre VALUES
  -> (301, 'Drama'),
  -> (302, 'Technology');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO MovieActor VALUES
  -> (1, 101),
  -> (1, 102),
  -> (2, 101);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> INSERT INTO MovieDirector VALUES
  -> (1, 201),
  -> (2, 202);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO MovieGenre VALUES
  -> (1, 301),
  -> (1, 302),
  -> (2, 302);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Display all tables
mysql> SELECT * FROM Movie;
+-----+-----+
| MovieID | MovieTitle |
+-----+-----+
| 1 | The Database Story |
| 2 | Digital World |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Actor;
+-----+-----+
| ActorID | ActorName |
+-----+-----+
| 101 | Arun Kumar |
| 102 | Priya |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Director;
+-----+-----+
| DirectorID | DirectorName |
+-----+-----+
|          201 | Ravi         |
|          202 | Kumar        |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Genre;
+-----+-----+
| GenreID | GenreName |
+-----+-----+
|        301 | Drama     |
|        302 | Technology |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM MovieActor;
+-----+-----+
| MovieID | ActorID |
+-----+-----+
|        1 |        101 |
|        2 |        101 |
|        1 |        102 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM MovieDirector;
+-----+-----+
| MovieID | DirectorID |
+-----+-----+
|        1 |          201 |
|        2 |          202 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM MovieGenre;
```

MovieID	GenreID
1	301
1	302
2	302

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- Reconstruct movie information
```

```
mysql> SELECT m.MovieID,  
-> m.MovieTitle,  
-> a.ActorName,  
-> d.DirectorName,  
-> g.GenreName  
-> FROM Movie m  
-> LEFT JOIN MovieActor ma ON m.MovieID = ma.MovieID  
-> LEFT JOIN Actor a ON ma.ActorID = a.ActorID  
-> LEFT JOIN MovieDirector md ON m.MovieID = md.MovieID  
-> LEFT JOIN Director d ON md.DirectorID = d.DirectorID  
-> LEFT JOIN MovieGenre mg ON m.MovieID = mg.MovieID  
-> LEFT JOIN Genre g ON mg.GenreID = g.GenreID  
-> ORDER BY m.MovieID;
```

MovieID	MovieTitle	ActorName	DirectorName	GenreName
1	The Database Story	Arun Kumar	Ravi	Drama
1	The Database Story	Arun Kumar	Ravi	Technology
1	The Database Story	Priya	Ravi	Drama
1	The Database Story	Priya	Ravi	Technology
2	Digital World	Arun Kumar	Kumar	Technology

```
5 rows in set (0.00 sec)
```